

Neural network stochastic differential equation models with applications to financial data forecasting

Luxuan Yang^a, Ting Gao^{a,*}, Yubin Lu^a, Jinqiao Duan^b, Tao Liu^c

^a School of Mathematics and Statistics & Center for Mathematical Sciences, Huazhong University of Science and Technology, Wuhan 430074, China

^b Department of Applied Mathematics, College of Computing, Illinois Institute of Technology, Chicago, IL 60616, USA

^c Securities Finance Department, China Securities Co., Ltd, China

ARTICLE INFO

Article history:

Received 6 April 2022

Revised 29 October 2022

Accepted 2 November 2022

Available online 6 November 2022

Keywords:

Stochastic differential equations

α -Stable Lévy motion

Neural network

Chaotic time series

ABSTRACT

In this article, we employ a collection of stochastic differential equations with drift and diffusion coefficients approximated by neural networks to predict the trend of chaotic time series which has big jump properties. Our contributions are, first, we propose a model called Lévy induced stochastic differential equation network, which explores compounded stochastic differential equations with α -stable Lévy motion to model complex time series data and solve the problem through neural network approximation. Second, we theoretically prove that the numerical solution through our algorithm converges in probability to the solution of corresponding stochastic differential equation, without curse of dimensionality. Finally, we illustrate our method by applying it to real financial time series data and find the accuracy increases through the use of non-Gaussian Lévy processes. We also present detailed comparisons in terms of data patterns, various models, different shapes of Lévy motion and the prediction lengths.

© 2022 Elsevier Inc. All rights reserved.

1. Introduction

In many scenarios, the past information that time series data contains can help people understand future phenomena. Time series forecasting is hereafter attractive in many research fields and lots of research work are developed to improve the prediction ability of the models. Traditionally, autoregressive models and moving average models are often applied to time series prediction. However, they essentially only capture linear relationship in the data set and can not afford many real cases, such as weather forecasting, financial market prediction or signal processing, which are often complex, nonlinear or even chaotic. Another class of statistical methods is based on Bayes' theorem in probability theory. Bayesian inference, as an applicable tool of Bayesian theory, is leveraged to deal with uncertainty in noisy environment and improve model accuracy by collecting some prior information. For example, Hummer [1] applies Bayesian inference to estimate the rate coefficients that can characterize the dynamic behavior of some diffusion models. Beltran-Villegas et al. [2] use the Bayesian inference method to obtain the position-dependent potential energy and diffusivity so as to fit particle dynamics. Sahoo and Patra [3] combine the Bayesian inference method with Markov Chain Monte Carlo (MCMC) algorithm to evaluate water pollution rates. With the help of some prior knowledge, this method can greatly improve the prediction accuracy. However, if the

* Corresponding author.

E-mail addresses: luxuan_yang@hust.edu.cn (L. Yang), tgao0716@hust.edu.cn (T. Gao), yubin_lu@hust.edu.cn (Y. Lu), duan@iit.edu (J. Duan), liutao@csc.com.cn (T. Liu).

prior distribution is not presumed correctly, the inference will go to the wrong direction. Hence, improving the reliability of prior knowledge is crucial to the model performance. As there always exists various kinds of uncertainties in the forecasting problems, how to accurately capture the intrinsic volatile nature of time series data is still challenging. For example, financial time series data could be unpredictable considering hidden factors, such as market structure, transaction behavior, etc., which are inextricably linked with the carbon price [4,5]. In addition, in most often the cases, missing data could occur in many industrial applications, taking the prediction of equipment's temperature time series for example [6]. To handle these kinds of complex data with uncertain prior knowledge, some chaotic theory is constructed for prediction where phase space reconstruction is one of the most important ideas. Considering the chaotic characteristic of stock time series, Zhong [7] improves short-term stock prediction by using Phase Space Reconstruction Theory combined with Recurrent Neural Network. Stergiou and Karakasis [8] treat Lyapunov time as the length of the neural network input to quantify the safe prediction horizon and use the current prediction as input to predict the next step. Prediction with rolling accumulated training samples is a coin of two sides. On the one hand, with more information put into the training set, it will help to better predict the trend in the next day. On the other hand, the accumulated error will also deviate the long term prediction from the correct track.

Recently, machine learning models outperform conventional methods in many aspects and have led a growing amount of researches to utilize various deep learning frameworks in the research of time series forecasting. One of the promising models, transformer [9], is bound to have a prominent prospect. Many variants of transformer models have emerged in the past two years, such as informer [10] and autoformer [11]. Informer proposes a novel probability based self-attention mechanism, which reduces the network size and improves long-term sequence prediction. In addition, the long sequence output is obtained through the generative decoder, which avoids the cumulative error propagation in the inference stage. Autoformer also boosts the performance of long-term sequence prediction by using the random process theory, which discards the self-attention mechanism of point-wise connections, and replaces with the autocorrelation mechanism of series-wise connections instead. As can be seen from the above, the self-attention mechanism has certain advantages for long-term prediction, but not has too much mathematical theories to explain how and why it works on various prediction time ranges. To this end, Macaron Transformer [12] establishes an analogy between the multi-particle dynamic system in physics and the model structure, which benefits the understanding of the intrinsic mechanisms of Transformer.

Nowadays, the marriage between deep learning and dynamical systems has become a topic of increasingly popularity, as they could benefiting each other in both theoretical and computational aspects. On the computational side, for example, Neural ODE [13] is a combination of residual network and ordinary differential equations. The output of the network is calculated with a black box differential equation solver by adjoint methods. The article also builds a time series signal generation model through the variational autoencoder (VAE) framework, and uses the neural network ODE as a part of it. For stochastic cases, neural stochastic differential equation (Neural SDE) [14] shows a continuous neural network framework based on stochastic differential equation with Brownian motions. Neural Jump Stochastic Differential Equations [15] also provides an approach to learn hybrid system with flow and jump. On the basis of differential equations, an innovative neural network based on stochastic differential equations, SDE-Net [16] captivates us. The core of the SDE-Net is to treat the deep neural network transformation as the state evolution of a stochastic dynamic system, and introduce the Brownian motion term to capture cognitive uncertainty. Moreover, there are also studies using neural networks based on physical regimes. The PINNs(Physics-Informed Neural Networks) [17] model just is a combination of mathematics and deep learning. Lu et al. [18] summarize PINNs(Physics-Informed Neural Networks) and analyze the feasibility of PINNs to solve partial differential equations. Chen et al. [19] propose a new framework based on PINNs(Physics-Informed Neural Networks) with a new loss function to solve inverse stochastic problems. O'Leary et al. [20] use stochastic physics-informed neural networks (SPINN) to learn the hidden physics dynamical systems with multiplicative noise. Kontolati et al. [21] propose a multi-scale model based on nonlinear PDEs consistent with a continuum theory, and apply manifold learning and surrogate-based optimization techniques to the framework of machine learning. Moreover, there are even articles that combine finance, mathematics and deep learning. Gonon and Schwab [22] prove that it is reasonable to use DNN in financial modeling of a large basket in a market with a jump.

On the theoretical part, since the neural network is a black box, people are seeking to explain some of its characteristics through numerical analysis of dynamical systems. For instance, Siegel and Xu [23] get the convergence rate of the two-layer neural network under different conditions with the mathematical tools of Fourier analysis. E et al. [24] define the Barron space and give direct and inverse approximation theorems holding for functions in the Barron space under two-layer neural network. Li et al. [25] pay attention to the deep residual neural network, which can be regarded as continuous-time dynamical systems and summarized some results on the approximation properties in continuous time. All these ideas have inspired us to prove the convergence of our proposed Lévy induced stochastic differential equation network (LDE-Net) model.

In many real world applications, Lévy noise plays an important part as a more general random fluctuation including complex non-Gaussian properties. Zhu [26] compares the Lévy noise with the standard Gaussian noise and concludes that the Lévy noise is more versatile and has a wider range of applications in such diverse areas as mathematical finance, financial economics, stochastic filtering, stochastic control, and quantum field theory [26]. Sangeetha and Mathiyalagan [27] indicate that Lévy motion also works with genetic regulatory networks. Considering the properties of Lévy motion, we choose it to better capture the uncertainty in financial data, such as abrupt transitions. Thus we introduce Lévy induced stochastic dynamical system to our neural network framework for modeling complex financial time series data.

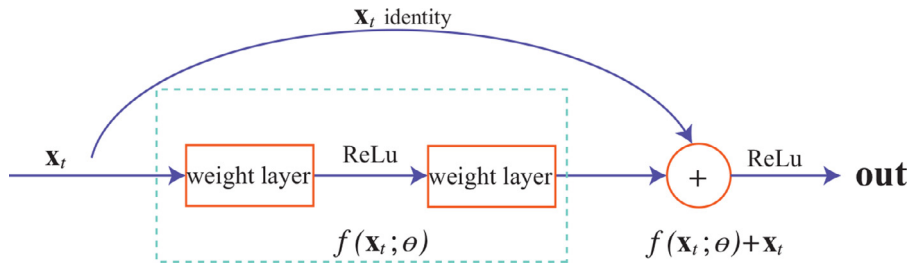


Fig. 1. Resnet Structure. (Fig. 2 in [31]).

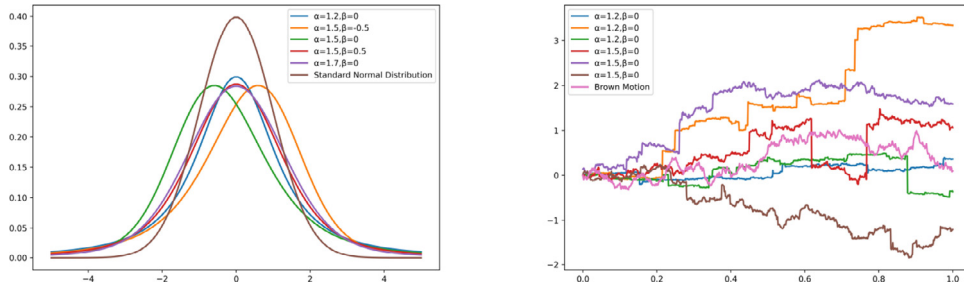


Fig. 2. left: Probability distribution functions of standard normal distribution and Lévy motion with different α and β . right: Brownian motion and Lévy motion with different α and β .

The rest of this paper is organized as follows. In Section 2, we present our neural network model based on stochastic differential equation induced by Lévy motion. The theoretical analysis of the model is given in Section 3. In Section 4, experimental results with real financial data are reported, which can help to evaluate the applicability and effectiveness of the model. Finally, we conclude the paper and give future research directions in Section 5.

Our main contributions are: First, we design an α -stable Lévy motion induced SDE (stochastic differential equation) to recognize model uncertainty. The experiment results prove that the use of non-Gaussian processes increases the accuracy of model predictions. Second, we prove that the approximated solution through our model (LDE-Net) converges in probability towards the true solution of the corresponding stochastic differential equation. Specially, this convergence bound avoids the curse of dimensionality since it is independent of state dimension. Third, we bring the attention mechanism into the model so that the proposed LDE-Net model can achieve the goal of multi-step prediction.

2. Methods

2.1. ResNet to ordinary differential equation

In the earlier days, linear autoregression models, such as AR, MA, ARIMA, etc., are often adopted to tackle with time series forecasting tasks. There are also some studies that use mathematical methods for time series forecasting. For instance, Wang et al. [28] apply data-driven ordinary differential equation to forecast daily PM2.5 concentration. Li et al. [29] propose a method based on the new bidirectional weakening buffer operator to improve the accuracy of time series forecasting. With the development of deep learning, a lot of neural network models such as RNN, LSTM, GRU, etc. are receiving more and more attention for various prediction problems. Because the structure of neural network lacks interpretability, people try to seek an explanation from mathematical theory. The emergence of residual neural networks (ResNet) gives a novel explanation on the accumulated error of neural networks through numerical analysis of ordinary differential equations.

Consequently, a growing amount of research arises in finding the relationship between neural network structure and discretized solution of numerical differential equations. For example, as neural network maps the input x to the output y through a series of hidden layers, the hidden representation can be regarded as the state of a dynamic system [16]. Lu et al. [30] confirmed that several neural network structures can be interpreted as various kinds of numerical discretizations of ordinary differential equations (ODEs). In particular, ResNet [31] can be regarded as a form of discretize Euler solution of ODEs. Here is the residual network structure in Fig. 1:

The mapping of $f(\mathbf{x}_t; \theta)$ can be realized by feedforward neural networks and the output of identity mapping $\mathbf{x}$ is added to the output of the stacked layers [31]. Therefore, the hidden state of the network can be described by the following equation:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + f(\mathbf{x}_t; \theta), \quad (2.1)$$

where $\mathbf{x}_t$ is the hidden state of the t th layer and $t \in \{0 \dots T\}$. If we add more layers and take smaller steps, the continuous dynamics of hidden units can be shown as follows [13]:

$$\frac{d\mathbf{x}_t}{dt} = f(\mathbf{x}_t; \theta) \quad (2.2)$$

Now we can see that ResNet can indeed match the Euler discretization of ordinary differential equations. This analogy can effectively help us understand the intrinsic behavior of ResNet.

2.2. Introduction to α -stable Lévy motion

In many real-world scientific and engineering applications, stochastic differential equations (SDE) models are widely used to investigate complex phenomena in various noisy systems. For financial market systems, SDE models can be used to quantify the set of optimal pricing barriers [32]. In addition, different financial models can be obtained by changing the form of the drift function. For instance, if the drift function $f(x) \equiv x$ in reflected SDE models, it can describe the price dynamics of the reference goods or service [33]. However, when $f(x) \equiv x^2$, the SDE models represent the risk-neutral term structure of the interest rate model [33]. In biological systems, SDE models can structurally identify the cell proliferation and death rates [34].

Moreover, stochastic differential equations also can model the complex data behaviors with potential uncertainty. SDE-Net [16] uses the stochastic differential equation with Brownian motion to replace the original hidden layer structure of ResNet. However, as many complex real world phenomena exhibit abrupt, intermittent or jumping behaviors, like impulses, using Brownian motion to capture data uncertainty may not be appropriate. Thus, we utilize a class of non-Gaussian noise induced stochastic differential equations with Lévy motion to improve the universality of predictive models. As we know, Lévy motion has drawn tremendous attention in various application, such as option pricing model [35], Arctic sea ice system [36], bearing degradation [37], climate change [38], agricultural product prices [39] etc.. Now, we use α -stable Lévy motion to capture cognitive uncertainty, which is the core of our proposed LDE-Net model.

Lévy Motion Let $X = (X(t), t \geq 0)$ be a stochastic process defined on a probability space $(\Omega, \mathcal{F}, P)$. We say that X is a Lévy motion if: (i) $X(0) = 0$ (a.s.); (ii) X has independent and stationary increments; (iii) X is stochastically continuous, i.e. for all $a > 0$ and all $s \geq 0$, $\lim_{t \rightarrow s} P(|X(t) - X(s)| > a) = 0$.

Symmetric α -Stable Random Variables $X \sim S_\alpha(\sigma, \beta, \gamma)$ is called a symmetric α -stable random variable if $\beta = 0$ and $\gamma = 0$, that is, $X \sim S_\alpha(\sigma, 0, 0)$. When $\sigma = 1$, it is called a standard symmetric α -stable random variable, and we denote this by $X \sim S_\alpha(1, 0, 0)$.

Symmetric α -Stable Lévy Motions A symmetric α -stable scalar Lévy motion L_t^α , with $0 < \alpha < 2$, is a stochastic process with the following properties: (i) $L_0^\alpha = 0$, a.s. (ii) L_t^α has independent increments. (iii) $L_t^\alpha - L_s^\alpha \sim S_\alpha((t-s)^{\frac{1}{\alpha}}, 0, 0)$. (iv) L_t^α has stochastically continuous sample paths, that is, for every $s > 0$, $L_t^\alpha \rightarrow L_s^\alpha$ in probability, as $t \rightarrow s$.

We add Lévy motion to the right side of Eq. (2.1) to make it the Euler discretization form of the stochastic differential equation. The continuous-time dynamics of the system (2.2) are then expressed as:

$$dx(t) = f(x(t); \theta_f)dt + g(x(t); \theta_g)dL_t^\alpha, \quad x(0) = x_0. \quad (2.3)$$

Here, the drift term $f(\cdot; \theta_f) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is a locally bounded Lipschitz continuous function. The diffusion matrix $g(\cdot; \theta_g) : \mathbb{R}^d \rightarrow \mathbb{R}^d \times \mathbb{R}^d$ is a diagonal matrix with the same diagonal elements. We then regard $g(\cdot; \theta_g)$ as a scalar function on $\mathbb{R}^d$ which is a locally bounded Lipschitz continuous function. The initial value $x_0 \in \mathbb{R}^d$ is a constant and $x(\cdot) : [0, T] \rightarrow \mathbb{R}^d$. Here L_t^α is a symmetric α -stable Lévy process in $\mathbb{R}^d$ with $1 < \alpha < 2$. Moreover, θ_f and θ_g are parameters of the two neural networks. Next, we will construct two neural networks for the above drift and diffusion coefficients and compound them in a discretized differential equation to accurately predict long-term time series.

2.3. LDE-Net

There are various numerical methods for solving differential equations with different accuracy, according to the expansion order of the unknown function. Considering that higher-order numerical methods will increase the computational cost and complicate the input structure, we use a simple Euler-Maruyama scheme with a fixed step size for efficient network training. Based on the Euler-Maruyama scheme, the approximation for Eq. (2.3) is

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \underbrace{f(\mathbf{x}_k; \theta_f)}_{\text{drift neural net}} \Delta t + \underbrace{g(\mathbf{x}_0; \theta_g)}_{\text{diffusion neural net}} (\Delta t)^{1/\alpha} L_k, \quad k = 0, 1, \dots, N-1. \quad (2.4)$$

To further explain Eq. (2.4), suppose the real data in our time series is collected on a fixed time interval T , that is, the whole time series is obtained at timestamps $[0, T, 2T, 3T, \dots]$. When predicting at each timestamp, the time interval T is divided into N sub-intervals and we get $\Delta t = \frac{T}{N}$. Now we consider the diffusion as N -step random walk with Lévy noise, here $L_k \sim S_\alpha(1, 0, 0)$ ($k = 0, 1, \dots, N-1$) are independent standard symmetric α -stable random variables, which are denoted as the last term in equation (2.4). This means we insert N discretized stochastic differential equations between every two

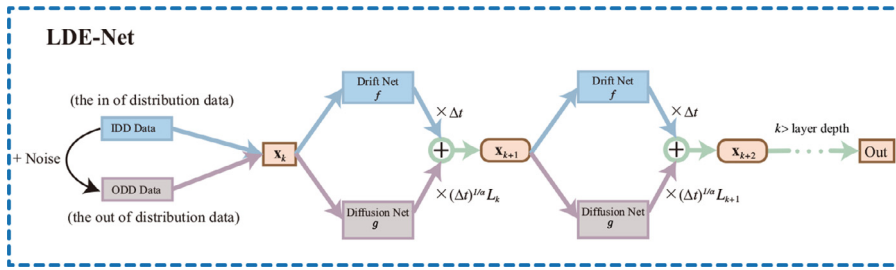


Fig. 3. Structure of LDE-Net.

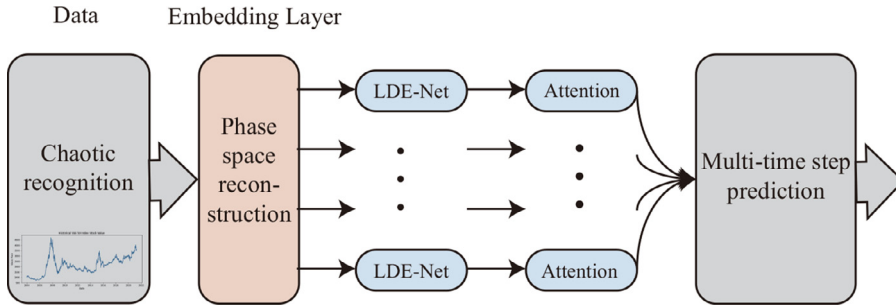


Fig. 4. Model construction.

consecutive data points. Here, $\mathbf{x}_0 = x_0$ is the initial input value of the neural network in $\mathbb{R}^d$ and $x(k\Delta t)$ is approximated by $\mathbf{x}_k$.

Moreover, both drift and diffusion coefficients in Eq. (2.4) are estimated by two different multi-layer perceptron (MLP) neural networks which are called the drift net and diffusion net respectively. In addition, we can also collect noisy data by adding noise to the training set. Then, we train the diffusion net through a binary classification problem using both the original training data and the noisy data. The drift net f and diffusion net g are devoted to capture the aleatoric uncertainty and epistemic uncertainty respectively. When applying the above networks into the iteration of Eq. (2.4), in the process of moving through the hidden layer, each of our network layers share the same parameters. Simultaneously, the output of the previous layer is transferred to the next layer as the input. And to reduce the computational burden, our diffusion coefficient is only determined by the initial value x_0 . When compounding this collection of discretized Euler iterations, we could predict the time series by the mean and variance of the final iteration x_N . The structure of our neural network architecture is shown in Fig. 3.

Based on the above settings, we have the training loss of our proposed model as follows:

$$\min_{\theta_f} E_{P_{\text{train}}} \text{Loss}(\mathbf{x}_N, y_{\text{true}}) + \min_{\theta_g} E_{P_{\text{train}}} g(\mathbf{x}_0; \theta_g) + \max_{\theta_g} E_{P_{\text{out}}} g(\tilde{\mathbf{x}}_0; \theta_g) \quad (2.5)$$

Here, the symbol P_{train} represents the distribution for original training set. P_{out} means the distribution of the training data with added noise (see Fig. 3). Hence, $\tilde{\mathbf{x}}_0 = \mathbf{x}_0 + \text{noise}$. Since the training process of the first term is a regression problem, we define $\text{Loss}(\mathbf{x}_N, y_{\text{true}})$ as the loglikelihood loss between the predicted output and true label y_{true} on training data. For the diffusion net, the weights are trained by the training data with and without added noise. We employ the Binary Cross Entropy as the loss functions in the last two items in Eq. (2.5).

Remark 1 Since we utilize non-Gaussian Lévy motion for modeling our neural network, we will show in the following sessions the better accuracy for time series data prediction both theoretically and experimentally.

Remark 2 To better generalize SDE-Net [16], we also use the attention mechanism to make the proposed model perform multi-step prediction in parallel.

Remark 3 This method actually learns $N (= T/\Delta t)$ pairs of different drift and diffusion coefficients, corresponding to $N (= T/\Delta t)$ different stochastic dynamical systems, where the former equation's output is the input of the next equation.

In summary, our model for the whole training process including data preparation is as follow: first, we check whether the time series data is chaotic or not by Lyapunov exponent (see Appendix A.1). Then if this is true, we will calculate intrinsic embedding dimension of input feature with phase space reconstruction (see Appendix A.2). Finally, we parallelly predict n days' label values for our training data through n different LDE-Nets with attention mechanism. The flow chart of the process is shown in Fig. 4.

3. Theoretical analysis

In this section, we will prove the convergence of our LDE-Net algorithm. First of all, the existence and uniqueness of the solution X_t to a stochastic differential equation with α -stable Lévy noise are proved in Theorem 3.1. The proof of the convergence follows three steps: First, under Assumption 1 and Lemma 3.1, we can get Lemma 3.2 so that we gain the mean uniform error bound of Euler-Maruyama method for stochastic differential equations with Lipschitz continuous drift coefficient by α -stable motion. Then, according to Lemma 3.3, any function in Barron space can be approximated by a two-layer neural network [24]. Moreover, stochastic gradient descent (SGD) can be proved to be convergent theoretically under some conditions [40]. Based on these two results, we give Assumptions 2 and 3. Under the assumptions and Lemma 3.4, we obtain that our neural networks satisfy the conditions in Proposition 3.1, which is crucial to the convergence proof of our LDE-Net model. Then, combining Lemma 3.2 and Proposition 3.1, we get Theorem 3.3, which shows that the approximated solution of our algorithm converges in probability to the solution of a corresponding stochastic differential equation.

First, we introduce Theorem 7.26 in Ch7 in [41] and Theorem 9.1 in Ch 4 in [42]. Here the function $b(\cdot)$ and $\sigma(\cdot)$ can be viewed as functions as well as two-layer neural networks.

Theorem 3.1. Let $\{\mathbf{U}, \mathcal{B}_{\mathbf{U}}\}$ be a measurable space and ν be a σ -finite measure on it. Let U_0 be a set in $\mathcal{B}_{\mathbf{U}}$ such that $\nu(\mathbf{U} \setminus U_0) < \infty$. If functions $b(\cdot)$ and $\sigma(\cdot)$ satisfy the Lipschitz condition:

$$\|b(x_1) - b(x_2)\|^2 + \int_{U_0} \|\sigma(x_1)z - \sigma(x_2)z\|^2 \nu(dz) \leq K|x_1 - x_2|^2, \quad x_1, x_2, z \in \mathbb{R}^d,$$

there exists a unique $(\mathcal{F}_t)$ -adapted right-continuous process $X(t)$ with left limits which satisfies equation

$$X(t) = X(0) + \int_0^t b(X(s))ds + \int_0^t \int_{\mathbf{U}} \sigma(X(s))u \tilde{N}(ds, du).$$

Here $\tilde{N}$ stands for the compensated Poisson measure. The measure ν is the Lévy measure of random poisson measure N .

Next, we will show the convergence of our model with respect to the number of neurons in hidden layer. In Eq. (2.4), the parameters θ_f and θ_g are only relevant to the width m of the neural networks. The parameter m is also called the number of neurons in the hidden layer. Hence, $f(\cdot; \theta_f)$ and $g(x_0; \theta_g)$ are obtained through neural network training. For simplicity, we use $f_m(\cdot)$ to represent $f(\cdot; \theta_f)$ and $g_m(x_0)$ to represent $g(x_0; \theta_g)$. Since x_0 is a constant, $g_m(x_0)$ is also a constant in $\mathbb{R}$. We rewrite Eq. (2.4) in the following form:

$$X_{k+1}^{(m)} = X_k^{(m)} + f_m(X_k^{(m)})\Delta t + g_m(X_0^{(m)})(\Delta t)^{1/\alpha}L_k, \quad X_0^{(m)} = x_0 \quad (3.1)$$

where $L_k \sim S_{\alpha}(1, 0, 0)$ ($k = 0, 1, \dots, N-1$). To emphasize that functions and variables are from neural network, we add the superscript (m) on them which also denotes the width of the neural network. Let $\Delta t \in (0, 1)$ and $N = \frac{T}{\Delta t}$. We also define the continuous version of Eq. (3.1) as follows:

$$X^{(m)}(t) = X^{(m)}(0) + \int_0^t f_m(\overline{X^{(m)}}(s))ds + g_m(X^{(m)}(0))L_t^{\alpha}, \quad X^{(m)}(0) = x_0, \quad (3.2)$$

where $\overline{X^{(m)}}(t) = X_k^{(m)}$ for $t \in [k\Delta t, (k+1)\Delta t)$ and L_t^{α} is a symmetric α -stable Lévy motion. This is similar to the method used in the proof of Liu [43]. Then, Eq. (2.3) can be correspondingly expressed by:

$$dx^{(m)}(t) = f_m(x^{(m)}(t))dt + g_m(x^{(m)}(0))dL_t^{\alpha}. \quad (3.3)$$

Where $x^{(m)}(0) = X^{(m)}(0) = x_0$ and x_0 is a constant. We use $\mathcal{B}$ and $\|\cdot\|_{\mathcal{B}}$ to denote the Barron space and Barron norm. $|\cdot|$ is the Euclidean norm.

Assumption 1. Assume that there exists a constant $K > 0$ such that

$$|f_m(x_1) - f_m(x_2)| \leq K|x_1 - x_2|,$$

for any $x_1, x_2 \in \mathbb{R}^d$ and $m \in \mathbb{N}^+$.

Note that Assumption 1 actually requires $\{f_m(\cdot)\}_{m \geq 1}$ to be uniformly Lipschitz continuous. Before proving Lemma 3.2, we are ready to state the following lemma.

Lemma 3.1. Suppose that Assumption 1 holds, for every $t \in [0, T]$, the mean absolute difference between the continuous version $X^{(m)}(t)$ and discrete version $\overline{X^{(m)}}(t)$ of Euler-Maruyama method is

$$\mathbb{E}|X^{(m)}(t) - \overline{X^{(m)}}(t)| \leq C_6(\Delta t)^{\frac{1}{\alpha}},$$

where constant $C_6 = 4K_1(1 + C_5) + 2|g_m(x_0)|C_3$ with $C_5 = C_4 + e^{6K_1T}$, $C_4 = 3(\mathbb{E}|x_0| + 2K_1T + C_3T^{\frac{1}{\alpha}}|g_m(x_0)|)$ and $K_1 = \max(K, f_m(x_0))$. Note that K is the Lipschitz constant for $f_m(\cdot; \theta_{f_m})$ and C_3 is a constant dependent on index α (see the exact expression of C_3 in [44]). The proof of Lemma 3.1 can be found in Lemma 3.3 in [43].

Lemma 3.2. Suppose that Assumption 1 holds, the mean uniform error bound of the Euler-Maruyama method for Eq. (3.3) satisfies

$$\mathbb{E} \sup_{0 \leq t \leq T} |x^{(m)}(t) - X^{(m)}(t)| \leq KC_6(\Delta t)^{\frac{1}{\alpha}} T \exp(KT),$$

where $x^{(m)}(t)$ is the solution of Eq. (3.3) and $X^{(m)}(t)$ is the continuous version of the corresponding Euler-Maruyama method in Eq. (3.2).

Proof of Lemma 3.2. For every $0 < \tau \leq t \leq T$,

$$\begin{aligned} |x^{(m)}(\tau) - X^{(m)}(\tau)| &= \left| \int_0^\tau [f_m(x^{(m)}(s)) - f_m(\overline{X^{(m)}}(s))] ds \right| \\ &\leq \int_0^t |f_m(x^{(m)}(s)) - f_m(X^{(m)}(s))| ds + \int_0^t |f_m(X^{(m)}(s)) - f_m(\overline{X^{(m)}}(s))| ds \\ &\leq K \int_0^t |x^{(m)}(s) - X^{(m)}(s)| ds + K \int_0^t |X^{(m)}(s) - \overline{X^{(m)}}(s)| ds \\ &\leq K \int_0^t \sup_{0 \leq r \leq s} |x^{(m)}(r) - X^{(m)}(r)| ds + K \int_0^T |X^{(m)}(s) - \overline{X^{(m)}}(s)| ds. \end{aligned}$$

Then, we get

$$\sup_{0 \leq \tau \leq t} |x^{(m)}(\tau) - X^{(m)}(\tau)| \leq K \int_0^t \sup_{0 \leq r \leq s} |x^{(m)}(r) - X^{(m)}(r)| ds + K \int_0^T |X^{(m)}(s) - \overline{X^{(m)}}(s)| ds.$$

Let $v(t) = \sup_{0 \leq r \leq t} |x^{(m)}(r) - X^{(m)}(r)|$. Then, taking expectations on both sides and by Fubini Theorem, we have

$$\mathbb{E}v(t) \leq K \int_0^t \mathbb{E}v(s) ds + K \int_0^T \mathbb{E}|X^{(m)}(s) - \overline{X^{(m)}}(s)| ds.$$

By Lemma 3.1, we can get

$$\mathbb{E}v(t) \leq K \int_0^t \mathbb{E}v(s) ds + KTC_6(\Delta t)^{\frac{1}{\alpha}}.$$

Using Gronwall's inequality and taking $t = T$, we get

$$\mathbb{E}v(T) \leq KC_6(\Delta t)^{\frac{1}{\alpha}} T \exp(KT),$$

which means

$$\mathbb{E} \sup_{0 \leq r \leq T} |x^{(m)}(r) - X^{(m)}(r)| \leq KC_6(\Delta t)^{\frac{1}{\alpha}} T \exp(KT).$$

□

Lemma 3.3. For every $f \in \mathcal{B}$ and constant $m > 0$, there exists a two-layer neural network $f_m(\cdot; \Theta)$, $f_m(\mathbf{x}; \Theta) = \sum_{k=1}^m \tilde{a}_k \sigma(\mathbf{b}_k^T \mathbf{x} + c_k)$, where $\tilde{a}_k = \frac{a_k}{m}$. (Θ denotes the parameters $\{(a_k, \mathbf{b}_k, c_k), k = 1, \dots, m\}$, such that

$$\|f(\cdot) - f_m(\cdot; \Theta)\|_{L^2}^2 \leq \frac{3\|f\|_{\mathcal{B}}^2}{m}.$$

Furthermore, we have

$$\|\Theta\|_{\mathcal{P}} := \sum_{j=1}^m |\tilde{a}_j| (\|\mathbf{b}_j\|_1 + |c_j|) \leq 2\|f\|_{\mathcal{B}}.$$

Here, m is the number of neurons in the hidden layers and $\{(\tilde{a}_k, \mathbf{b}_k, c_k), k = 1, \dots, m\}$ are the parameters of our neural network f_m . The proof of Lemma 3.3 can be found in Theorem 1 in [24].

Based on Lemma 3.3, two-layer neural networks in our model approximate function f in the following sense:

$$\|f_m(\cdot) - f(\cdot)\|_{L^2} \lesssim \frac{\|f\|_{\mathcal{B}}^2}{m^{\frac{1}{2}}} \leq C_1 m^{-\frac{1}{2}}, \quad (3.4)$$

where C_1 is a constant. Considering the input of the diffusion neural network g_m is always the initial value x_0 and the stochastic gradient descent (SGD) algorithm ensures its convergence [40], we assume that $g_m(x_0) \rightarrow g(x_0)$ when updating the parameters of the neural network.

Assumption 2. Assume that there is an underlying SDE:

$$Y(t) = Y(0) + \int_0^t f(Y(s)) ds + g(x_0) L_t^\alpha, \quad Y(0) = x_0, \quad (3.5)$$

where L_t^α is a symmetric α -stable Lévy motion with $1 < \alpha < 2$. Continuous function f and constant $g(x_0)$ satisfy the Lipschitz condition in Theorem 3.1.

Assumption 3. Assume for every compact domain $D \subset \mathbb{R}^d$, the neural networks trained functions $f_m(\cdot)$ are uniformly Lipschitz continuous and converge to $f(\cdot)$ in $L^2(D)$. Assume constants $g_m(x_0)$ converge to $g(x_0)$.

Based on Lemma 3.3, we can gain the convergence of $f_m(\cdot)$ in $L^2(D)$. Meanwhile, since the Barron norm of f is bounded, it ensures the uniformly Lipschitz continuity of $f_m(\cdot)$ on every compact sets D . In practice, we choose a sufficiently large D that real data varies within D to guarantee this assumption.

Here, Eq. (3.5) can be written as:

$$Y(t) = Y(0) + \int_0^t f(Y(s))ds + \int_0^t \int_{|u|>0} g(x_0)u\tilde{N}(ds, du), \quad Y(0) = x_0,$$

where $\tilde{N}(ds, du)$ is the Poisson random measure with compensator $ds \otimes \nu_\alpha(du)$. The α -stable Lévy measure ν_α is

$$\nu_\alpha(du) = \frac{1}{|u|^{1+\alpha}}du, \quad 1 < \alpha < 2.$$

Definition 3.2. Continuous functions $f_n(\cdot)$ locally uniformly converge to function $f(\cdot)$ if for every compact subset D in $\mathbb{R}^d$,

$$\sup_{z \in D} |f_n(z) - f(z)| \rightarrow 0, \quad n \rightarrow \infty.$$

Lemma 3.4. For a sequence of locally uniformly Lipschitz continuous functions $\eta_n(\cdot) \in L^2_{\text{Loc}}(\mathbb{R}^d \cap C(\mathbb{R}^d))$ and a locally Lipschitz continuous function $\eta(\cdot) \in L^2(\mathbb{R}^d)$, suppose for every compact domain $D \subset \mathbb{R}^d$,

$$\lim_{n \rightarrow \infty} \|\eta_n(\cdot) - \eta(\cdot)\|_{L^2(D)} = 0.$$

Then, functions $\eta_n(\cdot)$ also uniformly converge to function $\eta(\cdot)$ on every compact domain in $\mathbb{R}^d$.

Proof. Given a compact domain $D \subset \mathbb{R}^d$, we first prove that every subsequence of $\eta_n(\cdot)$ has a further subsequence that converges uniformly on the domain D .

For every subsequence of $\{\eta_n(\cdot)\}_{n \geq 1}$, still denoted by $\{\eta_n(\cdot)\}_{n \geq 1}$, we know that $\eta_n(\cdot)$ converges to $\eta(\cdot)$ in $L^2(D)$. Then there is a subsequence of $\eta_n(\cdot)$, denoted by $\eta_{n_k}(\cdot)$, that converges to $\eta(\cdot)$ for $z \in D \setminus D_0$. $D_0 \subset D$ is a set of Lebesgue measure zero.

For every $\epsilon > 0$, let $\delta = \epsilon/3L$, where L is the largest Lipschitz constant of functions $\eta_{n_k}(\cdot)$ and $\eta(\cdot)$. The open balls of radius δ , denoted by $\{B(z, \delta), z \in D \setminus D_0\}$, form an open cover of domain D . And there is a finite sub-cover of domain D , denoted by $\{B(z_n, \delta)\}_{1 \leq n \leq M}$. Let N be the largest number that for all $j > N$ and $\{z_n\}_{1 \leq n \leq M}$, $|\eta_{n_j}(z_n) - \eta(z_n)| < \epsilon/3$. Then, for every $z \in D$, there exists a point $z_l \in \{z_n\}_{1 \leq n \leq M}$ such that $\|z_l - z\| < \delta$. According to the uniform Lipschitz continuity of $\eta_{z_k}(\cdot)$ on domain D ,

$$|\eta_{n_j}(z) - \eta(z)| \leq |\eta_{n_j}(z) - \eta_{n_j}(z_l)| + |\eta_{n_j}(z_l) - \eta(z_l)| + |\eta(z_l) - \eta(z)| < \epsilon.$$

It means that the subsequence $\eta_{n_k}(\cdot)$ converges uniformly to $\eta(\cdot)$ on D .

Next, we will show that $\eta_n(\cdot)$ converges uniformly to $\eta(\cdot)$ on D with proof by contradiction.

Suppose there is $\tilde{\delta} > 0$ and for all $k > 0$, there exists $\tilde{z}_k \in D$ and $n_k > k$ such that

$$|\eta_{n_k}(\tilde{z}_k) - \eta(\tilde{z}_k)| > \tilde{\delta}.$$

For above k , $\{\eta_{n_k}(\cdot)\}_{k \geq 1}$ form a subsequence of $\{\eta_n(\cdot)\}_{n \geq 1}$. The previous argument shows the subsequence of $\{\eta_{n_k}(\cdot)\}_{k \geq 1}$ converges uniformly to $\eta(\cdot)$ on D , which contradicts with the assumption. As a result, we have proved that $\eta_n(\cdot)$ converges uniformly to $\eta(\cdot)$ on D , which means functions $\eta_n(\cdot)$ uniformly converge to function $\eta(\cdot)$ on every compact domain in $\mathbb{R}^d$. $\square$

Let $\mathbb{D}_{\mathbb{R}^d}[0, \infty)$ denote the Skorokhod space of functions from $[0, \infty)$ to $\mathbb{R}^d$. The following proposition is the cornerstone for proving the convergence of our model.

Proposition 3.1. For the d -dimensional approximating SDE

$$y_n(t) = y_n(0) + \int_0^t b_n(y_n(s))ds + \sigma_n L_t^\alpha, \quad y_n(0) = x_0, \quad (3.6)$$

where L_t^α is the d -dimensional symmetric α -stable Lévy process. Assume the d -dimensional functions $b_n(\cdot)$ satisfy the Lipschitz condition in Theorem 3.1 for all $n \geq 1$ and locally uniformly converge to $f(\cdot)$ of Eq. (3.5). Suppose the diagonal entries of $d \times d$ diagonal matrix $\{\sigma_n\}_{n \geq 1}$ converge to $g(x_0)$ of Eq. (3.5). Then the solution $y_n(\cdot)$ converge to $Y(\cdot)$ of Eq. (3.5) in probability with respect the Skorokhod distance in $\mathbb{D}_{\mathbb{R}^d}[0, \infty)$.

Proof. The proof relies on Theorem 5.4 and Corollary 5.6 of [45] and we will verify conditions C2.2(i) and C5.4 therein. Let

$$F_n(z, s) = \begin{pmatrix} b_n(z) & \sigma_n \end{pmatrix}, \quad Z_n(s) = \begin{pmatrix} s \\ I_s^\alpha \end{pmatrix} \quad \text{and} \quad U_n(s) = U(s) = x_0,$$

where Z_n are a $(d+1)$ -dimensional stochastic processes and F_n are $d \times (d+1)$ matrix-valued functions. Then Eq. (3.6) can be written in the following form,

$$y_n(t) = U(t) + \int_0^t F_n(y_n(s), s) dZ_n(s). \quad (3.7)$$

We first verify condition C2.2(i) of [45]. According to Lévy-Ito decomposition, we have

$$L_t^\alpha = \int_0^t \int_{|u| \leq 1} u \tilde{N}(ds, du) + \int_0^t \int_{|u| > 1} u N(ds, du) := M_t + A_t,$$

and Z_n has the following decomposition,

$$Z_n(t) = \begin{pmatrix} 0 \\ M_t \end{pmatrix} + \begin{pmatrix} t \\ A_t \end{pmatrix} := \tilde{M}_t + \tilde{A}_t.$$

We know process M_t is a martingale and A_t is of finite total variation. We denote the total variation of A_t by $T_t(A)$. The quadratic variation of M_t , denoted by $[M]_t$, and its expectation are

$$[M]_t = \int_0^t \int_{|u| \leq 1} |u|^2 N(ds, du) \quad \text{and} \quad \mathbb{E}([M]_t) = \int_0^t \int_{|u| \leq 1} |u|^2 ds \nu_\alpha(du) < \infty.$$

It means that for every stopping time τ and time t , we have

$$\mathbb{E}([\tilde{M}]_{t \wedge \tau} + T_{t \wedge \tau}(\tilde{A})) \leq \mathbb{E}([M]_t + T_t(A) + t) < \infty.$$

Then, condition C2.2(i) is satisfied. Next, we will verify condition C5.4.

Let $T_1[0, \infty)$ denote the collection of non-decreasing mappings λ of $[0, \infty)$ onto $[0, \infty)$ satisfying $\lambda(t+h) - \lambda(t) \leq h$ with respect to all $t, h \geq 0$. By Example 5.3 of [45], we know continuous functions $F_n(\cdot)$ and $F(\cdot)$ define mappings G_n and G through

$$G_n(\xi, \lambda, s) = F_n(\xi(\lambda(s))), \quad G(\xi, \lambda, s) = F(\xi(\lambda(s))), \quad \lambda \in T_1[0, \infty),$$

where function $\xi \in \mathbb{D}_{\mathbb{R}(d+1)}[0, \infty)$. With a given compact set $\mathcal{H} \subset D_{\mathbb{R}^{d+1}}[0, \infty) \times T_1[0, \infty)$, we know the projections

$$\mathcal{H}_1 = \{\xi : (\xi, \lambda) \in \mathcal{H}\}, \quad \mathcal{H}_2 = \{\lambda : (\xi, \lambda) \in \mathcal{H}\}$$

are compact sets in $D_{\mathbb{R}^{d+1}}[0, \infty)$ and $T_1[0, \infty)$ respectively. Then, for every $t > 0$, there exists $M > 0$ such that,

$$\sup_{(\xi, \lambda) \in \mathcal{H}} \sup_{s \leq t} (|\xi(\lambda(s))| + |\lambda(s)|) \leq M.$$

As a result, we have

$$\begin{aligned} \sup_{(\xi, \lambda) \in \mathcal{H}} \sup_{s \leq t} |G_n(\xi, \lambda, s) - G(\xi, \lambda, s)| &= \sup_{(\xi, \lambda) \in \mathcal{H}} \sup_{s \leq t} (|b_n(\xi(\lambda(s))) - b(\xi(\lambda(s)))| + |\sigma_n - \sigma|) \\ &\leq \sup_{|u| \leq M} (|b_n(u) - b(u)| + |g_n - g|) \rightarrow 0, \quad \text{as } n \rightarrow \infty. \end{aligned}$$

Condition C5.4(i) of [45] is then verified. Condition C5.4(ii) is a direct result of the continuity of functions F_n and condition C5.4 is verified. By Theorem 3.1, we know that solution Y to Eq. (3.5) exists uniquely. Then, Corollary 5.6 in [45] can be applied to ensure that solutions y_n of Eq. (3.6) converge in probability to solution Y of Eq. (3.5) with respect to the Skorokhod distance in $\mathbb{D}_{\mathbb{R}^d}[0, \infty)$. $\square$

The following theorem is the main result of this section. It proves the convergence of our model in d -dimension.

Theorem 3.3. Suppose the two-layer neural network trained d -dimensional functions $f_m(\cdot)$ and $d \times d$ diagonal matrices $g_m(x_0)$ satisfy the Assumptions 1 and 3. Assume further the Assumption 2. Then, the continuous dynamics of our LDE-Net model $X^{(m)}$ converge in probability to the solution Y of Eq. (3.5).

Proof. According to Lemma 3.2, for $X^{(m)}$ and $x^{(m)}$ defined in (3.2) and (3.3) respectively, we have

$$\mathbb{E}(d_T(X^{(m)}, x^{(m)})) \leq \mathbb{E}\left(\sup_{0 \leq t \leq T} |X^{(m)} - x^{(m)}|\right) \rightarrow 0, \quad \text{as } \Delta t \rightarrow 0,$$

where distance d_T is the distance in the Skorokhod space $\mathbb{D}_{\mathbb{R}^d}[0, T]$. It means that $X^{(m)}$ converge to $x^{(m)}$ in probability.

With Assumption 3 and Lemma 3.4, we know continuous functions $f_m(\cdot)$ converge to $f(\cdot)$ on every compact set $D \subset \mathbb{R}^d$. Take $f_m(\cdot)$ and $g_m(x_0)$ as the drift term $b_n(\cdot)$ and the diffusion term σ_n respectively in Eq. (3.6). Then, functions $f_m(\cdot)$ satisfy the condition proposed for the drift term $b_n(\cdot)$ in Proposition 3.1. As constants $g_m(x_0)$ are bounded and also converge to

$g(x_0)$, by Proposition 3.1, we know that solution $x^{(m)}$ of Eq. (3.3) converges in probability to solution Y of Eq. (3.5) with respect to the Skorokhod distance in $\mathbb{D}_{\mathbb{R}^d}[0, T]$. Then we know for every $\delta > 0$,

$$\mathbb{P}(d_T(Y, X^{(m)}) > \delta) \leq \mathbb{P}(d_T(X^{(m)}, x^{(m)}) > \frac{\delta}{2}) + \mathbb{P}(d_T(Y, x^{(m)}) > \frac{\delta}{2}) \rightarrow 0, \quad \text{as } m \rightarrow \infty \text{ and } \Delta t \rightarrow 0.$$

Then, we have proved that $X^{(m)}$ converge in probability to the solution Y of Eq. (3.5) with respect the Skorokhod distance in $\mathbb{D}_{\mathbb{R}^d}[0, T]$. $\square$

4. Experiments

In this section, we first preprocess the real data through Phase Space Reconstruction with Cao's method in Section 4.1 (see Appendix A). After that, we compare our model performance under various scenarios from Sec 4.2 to Section 4.5. To be specific, in order to bring the real data into our LDE-Net model, we find the optimal delay time and embedding dimension for the input vector in Section 4.1, and then show the prediction accuracy for three different stock trending patterns in Section 4.2. Furthermore, we choose two appropriate patterns of them to compare our model with several baselines on the same test data with the same evaluation measurements in the following sections. In addition, we also numerically analyze the effect of different α values in Lévy motion and forecasting days on prediction accuracy. The code of all experiments has been open sourced on our GitHub repository: <https://github.com/senyuanaya/LDENet>.

4.1. Dataset

Time series in many real scenarios usually appear to be in irregular and hard to understand, so it is important to uncover the governing laws hidden under a non-linear mapping. Chaotic time series can be exactly a type of sequence that restores its original system through mapping. The chaotic theory of nonlinear complex dynamics science holds that: all realistic nonlinear systems are in dynamic evolution, and may present periodic ordered states and non-periodic disordered states (that is, chaotic states) during their evolution [46]. Tian and Hao [5] just employs phase space reconstruction to reconstruct the carbon price data for feature selection. Stergiou and Karakasidis [8] takes Lyapunov time as the safe horizon and compares the long-term prediction effects of four different neural network models inside and outside the safe horizon. It was found that even outside the safe horizon, high accuracy can also be obtained. Considering the effectiveness of this methodology, we also use phase space reconstruction to effectively extract data feature representation by viewing the dynamics of monitoring data.

Due to the importance of the financial market, which is related to national economic development and social stability, we select stock data as time series. Certainly, our approach is still applicable to time series data such as weather and electricity. In this paper, we consider different types of stock data and divide them separately into a training set and a testing set according to the ratio of 4 : 1. Then we use the Wolf method [47] to calculate the maximum Lyapunov exponent of data and determine whether the financial time series is chaotic. As for the embedding time delay τ , the autocorrelation function or mutual information [48] is a universal and effective method for the stationary series. However, according to Zbilut [49] and Yao and Lin [50], choosing a lag value $\tau = 1$ is usually appropriate for non-stationary data like financial time series. So we also choose the delay time $\tau = 1$. Next, we analyze the effect of the embedding dimension on the maximum Lyapunov exponent. Then, we calculate the best embedding dimension of the stock data and reconstruct the state space with the Cao's method [51]. Finally, we use the reconstructed data as the input of the LDE-Net model and perform a multi-step prediction, that is, using M (the best embedding dimension) days of data prior to the prediction date and forecast the trend for the next four days. In addition, we compare our proposed model with some other different models and find out the impact of different α on the prediction.

Three types of stock patterns

According to the different fluctuations of data sets, we select three stocks corresponding to SSE Energy Index, SSE50 Index, and SSE Consumer Index. For SSE Energy Index, it fluctuates greatly on the training set, but behaves relatively smoothly on the testing set. For SSE50 Index, it shows large fluctuations on the training set and relatively small on the testing set. For SSE Consumer Index, it performs relatively smoothly on the training set but fluctuates greatly on the testing set. Furthermore, we select SSE Energy index with a total sample of 3086, SSE50 Index with a total sample of 4194 and SSE Consumer Index with a total sample of 2497 separately from 2009 to 2021, 2004–2021, and 2011–2021. (See Fig. 5)

Lyapunov exponent and intrinsic embedding dimension

After completing stock selection, we need to determine the input structure. This requires the use of the phase space reconstruction technique we mentioned earlier. Considering that stock data cannot involve future information in practical applications, we choose training data to identify chaotic characteristics and determine embedding dimensions. Therefore, the sample sizes used for phase space reconstruction for the above three stocks are 2469, 3356, and 1998 respectively. According to the calculation method of the maximum Lyapunov exponent (see Appendix A.1), we conclude that the three stocks are all chaotic time series. Moreover, we also show the safe prediction horizons [8] according to Lyapunov time in Table 1.

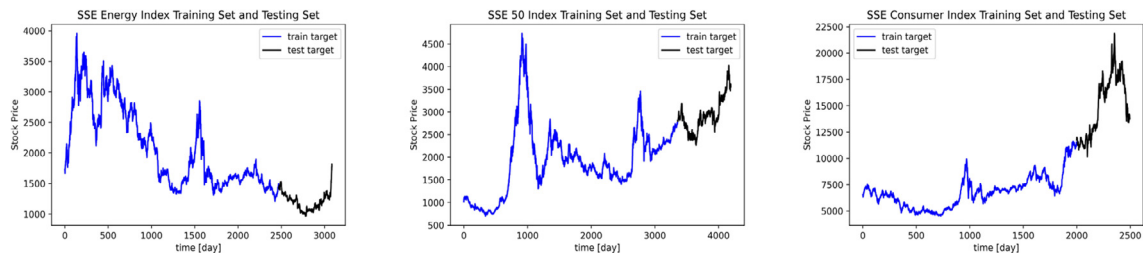


Fig. 5. left: Data set of SSE Energy index. middle: Data set of SSE 50 index. right: Data set of SSE Consumer Index.

Table 1
Phase Space Reconstruction Indicators.

Data Set	Maximum Lyapunov Exponent	Lyapunov Time (Safe Prediction Horizon)
SSE Energy Index	0.0231	43
SSE50 Index	0.0242	41
SSE Consumer Index	0.0324	30

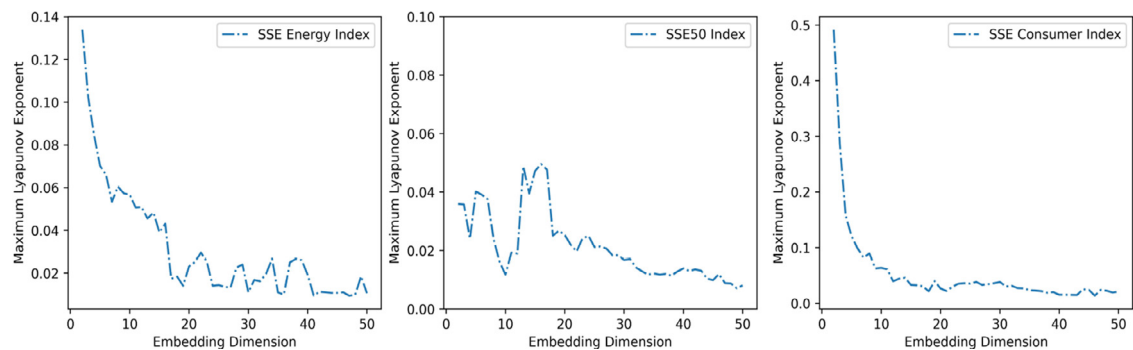


Fig. 6. Relationship between embedding dimensions and maximum Lyapunov exponent. (left: SSE Energy Index, middle: SSE50 Index, right: SSE Consumer Index).

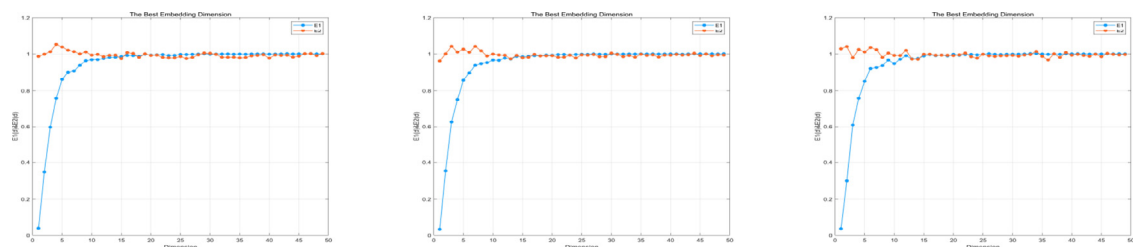


Fig. 7. The Best Embedding Dimensions. (left: SSE Energy Index, middle: SSE50 Index, right: SSE Consumer Index).

Subsequently, we explore the correlation between the maximum Lyapunov exponent and embedding dimension under the condition of fixed delay time. From Fig. 6, all the maximum Lyapunov exponents have descending trend and tend to be stable with the increment of embedding dimension. This indicates that the maximum Lyapunov exponent will converge over the embedding dimension. Frank explains that this phenomenon is caused by the fact that high-dimensional embedding can simultaneously reduce and diffuse the neighbor distribution, resulting in a reduced Lyapunov exponent estimation [52]. To determine the optimal embedding dimension when the maximum Lyapunov exponent plateaus, we choose Cao's method (see Appendix A.2). After calculation, the optimal embedding dimensions for the three stocks are: $m_1 = 20$, $m_2 = 23$, $m_3 = 15$ respectively (Fig. 7). Based on these results, we obtain the input data structure. Moreover, considering that Lyapunov time can be regarded as the safe prediction horizon [8], we take the safe horizon as the benchmark input size and compare the performance of models inside and outside this benchmark. From the tables in Appendix B.3 (Table B.1, Table B.2), we can know that our model has good prediction performance even outside the safe horizon, which is similar to the result in [8].

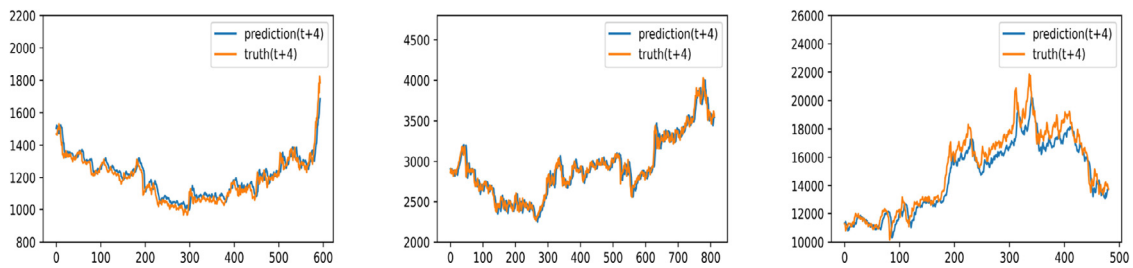


Fig. 8. The prediction effect of different stocks on the fourth day. (left:SSE Energy Index, middle: SSE50 Index, right: SSE Consumer Index).

Table 2

Forecast results of different stock ($\alpha = 1.5$).

Data	MSE(t+1)	MSE(t+2)	MSE(t+3)	MSE(t+4)
SSE Energy Index	0.0010	0.0022	0.0034	0.0045
SSE50 Index	0.0029	0.0058	0.0091	0.0125
SSE Consumer Index	0.2024	0.2210	0.1951	0.3301

Table 3

Forecast results of different models.

Data	Model	MSE	RMSE	MAE
SSE Energy Index	ARIMA	0.0188	0.1371	0.0945
	LSTM	0.0054	0.0735	0.0597
	SDE-Net	0.0011	0.0335	0.0256
	LDE-Net	0.0009	0.0295	0.0209
SSE 50 Index	ARIMA	0.0113	0.1064	0.0767
	LSTM	0.0031	0.0554	0.0408
	SDE-Net	0.0030	0.0546	0.0394
	LDE-Net	0.0029	0.0537	0.0386

4.2. Experiment 1: Forecasting with different stock patterns

When comparing different stock trend types, we only show the forecasting effect of the fourth day under the same $\alpha = 1.5$ (Fig. 8). By observing the results of loss (Table 2), we can determine which type of stock data our model is more suitable for. We can observe the prediction effect of four days in Appendix Fig. B.1.

As can be seen from Fig. 8, the prediction effect of the SSE Consumer Index is the worst and the LDE-Net model is suitable for predicting the future value of stocks with large fluctuations in historical data. When the training set fluctuates more frequently with larger amplitude, Lévy noise can better capture the uncertainty in the prediction process, so as to achieve more accurate prediction. However, historical data in Consumer Index flatten while the prediction set has bigger volatility, which is just the opposite of the trend situation described earlier. Therefore, the fitting effect between the predicted result and the true value is not ideal for Consumer Index. In the follow-up part, we will choose SSE Energy Index and SSE50 Index for experimentation to confirm the significance of LDE-Net.

4.3. Experiment 2: Forecasting with different models

Under the condition of setting $\alpha = 1.5$, we compare the proposed model with the traditional SDE-Net [16] and commonly used forecasting models for stock data to check whether the proposed model makes sense. We have chosen LSTM [53] and ARIMA [54], for common models. Since the accuracy of forecasting one day is the highest, we only compare the effect of forecasting one day. Here we only show the prediction effect images of the SSE Energy Index with the same coordinate scales. For the SSE 50 Index, the results can be found in Fig. B.2.

From the above figures (Fig. 9) only for SSE Energy Index, we can intuitively find that the prediction effect of LDE-Net is the best. The predicted values of LSTM and SDE-Net in the next 200–300 days (testing set) are obviously not close to the true values. Although the predicted values of ARIMA seem to fit very well overall, it can be seen that the first few real values are not close to the predicted values. We select the first 50 data of the testing set and refine the coordinate scale to zoom in and observe the prediction effect. It can be found that the curve fitted by ARIMA has a significant time lag. Therefore, ARIMA seems to be accurate on the surface, but it is a visual error caused by the large coordinate scale. To see the difference more specifically, we give a comparison table of loss defined by MSE, RMSE, and MAE for the SSE Energy Index and SSE 50 Index.

From the three indicators in Table 3, it is verified that the conclusions we have obtained based on the images are correct. For the two types of stock representative data, the prediction effect of our LDE-Net is better than other models. For SSE

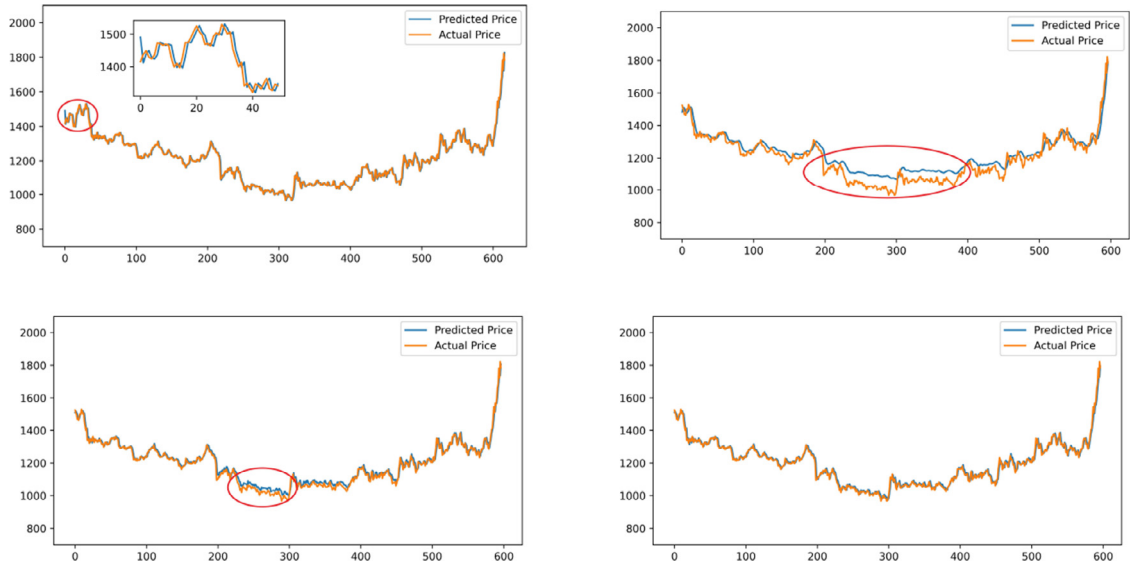


Fig. 9. For SSE Energy Index, the predictive effects of different models. (The upper left is ARIMA, the upper right is LSTM, the lower left is SDE-Net, and the lower right is LDE-Net.).

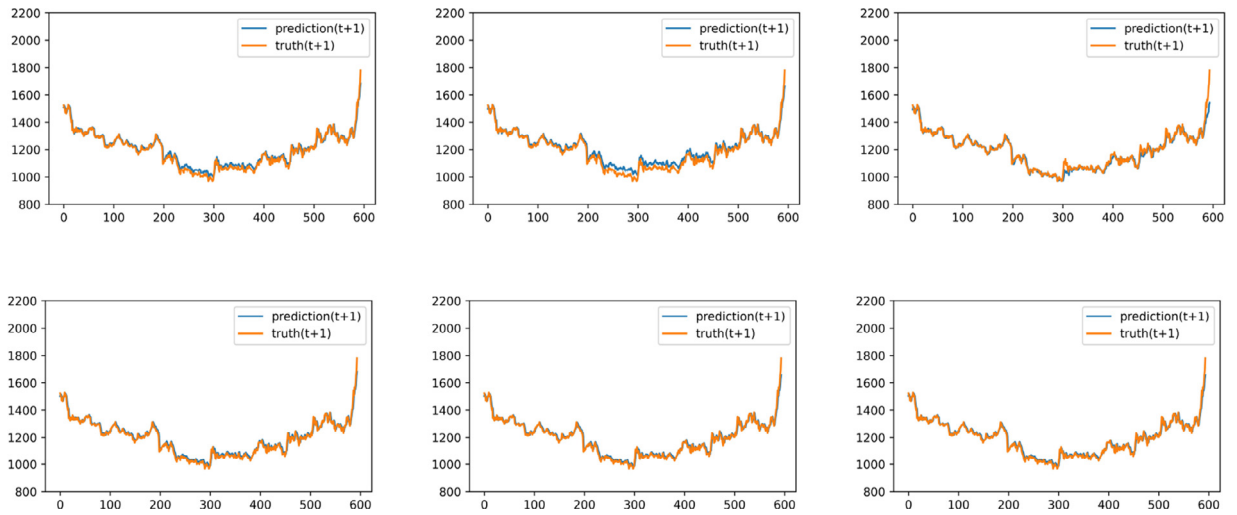


Fig. 10. For SSE energy index, the predictive effect of different α . (The upper left is $\alpha = 1.2$, the upper middle is $\alpha = 1.3$, the upper right is $\alpha = 1.4$, the lower left is $\alpha = 1.5$, the lower middle is $\alpha = 1.6$, and the lower right is $\alpha = 1.8$).

Energy Index, the MSE value is about 0.08672%, and the accuracy is 29% higher than the original SED-Net. In addition, RMSE is about 2.945%, and the error is 0.401% less than SED-Net. Also, MAE is reduced by about 0.47%. For SSE 50 Index, the values of RMSE and MAE are about 0.1% less than SED-Net. In general, the proposal of LDE-Net has a certain meaning.

4.4. Experiment 3: Forecasting with different α

Considering that different α will produce different jumps, we want to observe the effect of model predictive ability without a fixed $\alpha=1.5$. Therefore, we input the reconstructed data into the LDE-Net model with $\alpha = 1.2, 1.3, 1.4, 1.5, 1.6, 1.8$ for the same stock (Fig. 10).

From the results of the MSE value in Table 4, it can be seen that the value of α can have a certain influence on the prediction of the model. For example, for the same stock, it can be seen that the best α of the SSE Energy Index is 1.6, while the best α of the SSE 50 Index is 1.5. For different stocks, the greater the data fluctuation, the smaller α value of the stock, which is in line with the nature of Lévy motion that is smaller α can give greater jump.

Table 4
Forecast results of different α .

Data	Value of α	MSE(t+1)1.0*1e-3	MSE(t+2)1.0*1e-3	MSE(t+3)1.0*1e-3	MSE(t+4)1.0*1e-3
SSE Energy Index	1.2	1.1555	2.8272	3.9941	5.2052
	1.3	2.0929	2.2900	4.1513	5.3823
	1.4	1.7146	2.4714	3.1843	4.6635
	1.5	1.0317	2.2017	3.3650	4.5471
	1.6	0.9455	2.1151	2.8728	5.4028
	1.8	0.9890	2.0429	2.8211	5.5467
SSE50 Index	1.2	3.2909	6.2504	9.3327	13.3091
	1.3	3.1861	6.0212	9.6572	12.8533
	1.4	2.9535	5.9002	9.4551	12.8543
	1.5	2.9415	5.8059	9.1026	12.4943
	1.6	2.9809	6.0121	9.6383	12.8007
	1.8	2.9804	6.0358	9.4742	12.6342

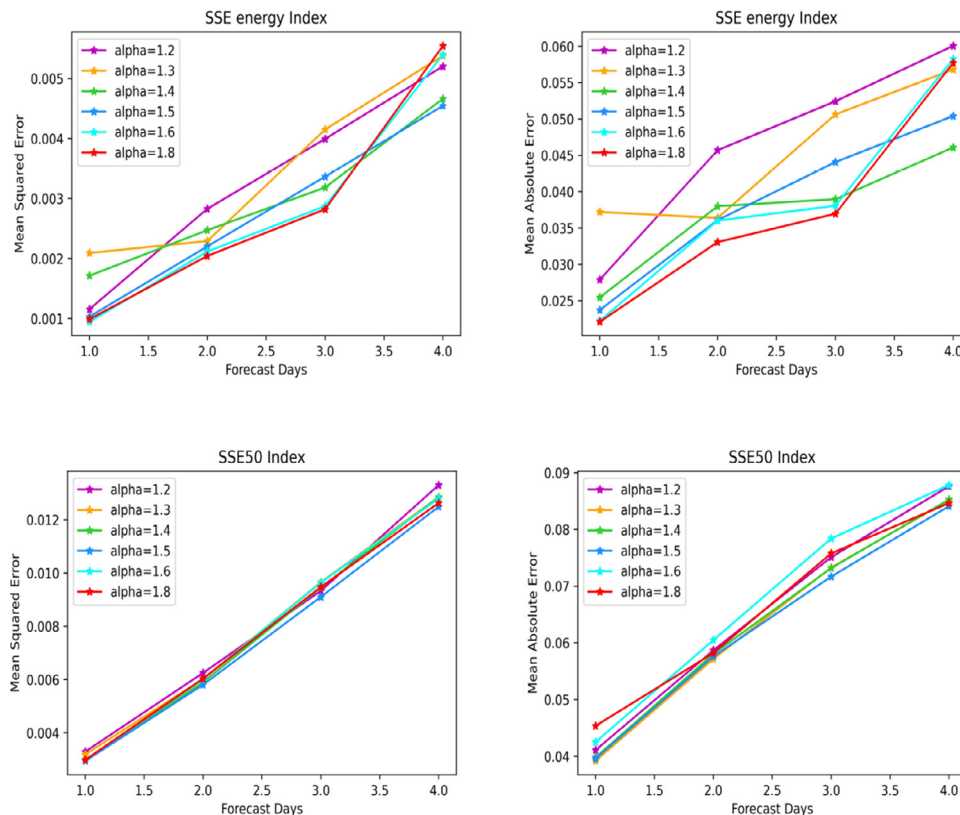


Fig. 11. The prediction error rate over forecasting days: mean squared error of SSE Energy Index (upper left), the mean absolute error of SSE Energy Index (upper right), mean squared error of SSE50 Index (lower left), mean absolute error of SSE50 Index (lower right).

4.5. Experiment 4: Effect of forecasting days on prediction accuracy

We also investigate the order of accuracy over predicting days and get some interesting results. For example, the error is roughly proportional to the length of forecasting days (Fig. 11), even if we change the value of α . This linear relationship indicates that the error rate (increment of error vs. prediction time) is of $O(1)$. For longer forecasting days such as ten days, the error still has the same phenomenon (See Fig. B.3 and Table B.3 in Appendix B.4). Moreover, from the figures (Figs. B.4, B.5), we can see that there will be a prediction shift with longer forecasting days. Hence, long-term prediction is our future improvement direction.

5. Conclusions and discussion

In this paper, we present a series of latent stochastic differential equations equipped with neural networks to forecast financial data, which is non-stationary with noise. The special properties of α -stable Lévy motion induced stochastic differ-

ential equations could help us to capture data and model uncertainty in a better way. To effectively utilize the information of real financial data, on the one hand, we explore its chaotic behavior and obtain the optimal intrinsic dimension of our input training data. And on the other hand, we use the attention mechanism to perform parallel operations on multiple models to achieve multi-step forecasting. All in all, we conclude our main findings in the following: First, the model LDE-Net we proposed is more suitable for predicting time series data that fluctuates with high volatility, as α -stable Lévy motion can better capture the training data trend with epistemic jumps. Second, we have theoretically proved the convergence of our algorithm, without curse of dimensionality. Finally, we also illustrated that the order of accuracy linearly changes over forecasting days.

Due to the fact that our model can better capture the underlying information in the case of historical data with jumps, LDE-Net algorithm with high accuracy can be applied to various chaotic time series predictions, such as index stocks, agricultural product prices, electricity load, climate change, remaining useful life prediction in bearing degradation, etc. For financial data, it can help investors to select the best portfolio investment as well as some indicators for timing strategies. In addition, with multi-step forecasting in our model, people can obtain long-term information.

For future research work, one aspect can be optimizing the selection of α values and another direction is to estimate generalization error bound in terms of data complexity. Simultaneously, considering that self-attention has a good performance in long-term prediction, we hope it can improve our attention mechanism. In addition, the application of the adjoint sensitivity method to stochastic differential equations is also worthy of further consideration. Since the discretized Euler method is relatively rough, the neural ordinary differential equation (Neural ODE) applies the ODE Solver to the adjoint system, which can both reduce computational cost and improve model accuracy efficiently. Considering the advantages of ODE Solver and the benefits of time continuation, it might be interesting to extend our current model into a continuous version under a similar framework of Neural ODE. As in the neural stochastic differential equation (Neural SDE), the adjoint stochastic differential equation is constructed under the assumption of Stratonovich noise, which is easier compared with the Itô case. Therefore, for stochastic differential equations induced by Lévy motion, it is novel to extend the adjoint sensitivity method correspondingly.

Our method has similar leanings on drift coefficient and diffusion coefficient like the work of stochastic physics-informed neural networks (SPINN), yet they have different goals. The former one is aiming for prediction with implicit compounded drift and diffusion coefficients while the latter can identify an unknown physical system explicitly. Therefore, it is interesting to learn an inverse problem through the framework of our model.

Data Availability

Data will be made available on request.

Acknowledgments

This work was supported by [National Natural Science Foundation of China \(NSFC\) 12141107](#). We would like to thank Wei Wei, Zibo Wang, Ao Zhang and Huifang Huang for helpful discussion. We also thank the editor, the associated editor and the anonymous reviewers for numerous constructive suggestions which help improve the presentation of this article.

Appendix A

A1. Lyapunov exponent

Based on Takens Theorem [55], we know that chaotic time series can restore the original system to establish a prediction model. Therefore, identifying the chaos of time series is the basis of prediction. There are many ways to identify whether a time series is chaotic, such as the largest Lyapunov exponent method, power spectrum method, system phase diagram structure method etc. Here we use the largest Lyapunov exponent method.

The Lyapunov exponent can describe the characteristics of the dynamics of the system. Assuming if that the value of Lyapunov is represented by λ , we have three cases: (i) $\lambda < 0$, the motion state of the system tends to be stable and is not sensitive to the initial state of the system at this time. (ii) $\lambda = 0$, the system is in a stable state. (iii) $\lambda > 0$, the system movement will enter a chaotic state, and the corresponding mapping is called chaotic mapping. Therefore, judging the magnitude and sign of the Lyapunov exponent becomes a criterion for whether the system enters chaos. The calculation methods of the maximum Lyapunov exponent mainly include definition method, Wolf method [47], orthogonal method and small number method. For unknown nonlinear dynamic systems, we use the wolf method of orbit tracking to solve the maximum Lyapunov exponent of the sequence.

Wolf method

Since we cannot find the exact differential equation of the original dynamic system, we only calculate the maximum Lyapunov exponent through univariate time series by using analysis and orbit tracking methods. In 1985, a method to estimate

the maximum Lyapunov exponent was proposed [47] based on the evolution of phase trajectories, phase planes, and phase volumes. This method is based on orbit tracking which is widely used in the prediction of chaotic time series.

Assuming that the chaotic time series is $x_1, x_2, \dots, x_n$, the embedding dimension is m , and the delay time is τ , then the reconstructed phase space is

$$Y(t_i) = (x(t_i), x(t_{i+\tau}), \dots, x(t_{i+(m-1)\tau})), \quad i = 1, 2, \dots, N. \quad (\text{A.1})$$

Taking the initial point as $Y(t_0)$ and the nearest point of it as $Y_0(t_0)$, we define L_0 as the distance between them. Then, we track the evolution of the distance between these two points over time until L_0 reaches the critical value ε ($\varepsilon > 0$) at t_1 , that is $L_1 = |Y(t_1) - Y_0(t_1)| > \varepsilon$. Similarly, we keep $Y(t_1)$ and find its nearest point $Y_1(t_1)$. We set $L'_1 = |Y(t_1) - Y_1(t_1)| < \varepsilon$ and hope that the angle of L_1 and L'_1 is as small as possible. Continue the above process until $Y(t)$ reaches the end of the time series N . The total number of iterations of this process to track the evolution is M . Then, the maximum Lyapunov exponent is :

$$\lambda = \frac{1}{t_M - t_0} \sum_{i=1}^M \ln\left(\frac{L'_i}{L_i - 1}\right). \quad (\text{A.2})$$

If the maximum Lyapunov exponent is positive, we can get the Lyapunov time h :

$$h = 1/\lambda \quad (\text{A.3})$$

Since the Lyapunov time is the limit of the predictability of the system, it can be regarded as the safe predication horizon.

A2. Feature embedding with phase space reconstruction

After showing that the time series has chaotic behavior by the maximum Lyapunov exponent, we need to represent our feature embedding. The phase space reconstruction technology has two key parameters: the embedded dimension and the delay time. We assume that the time delay τ is not related to the reconstruction dimension m during reconstruction. Since the current moment is usually related to the information at the previous moment, we choose the time delay $\tau = 1$ to ensure the integrity of the information. The next step is to calculate its embedding dimension.

Due to the fact that the chaotic sequence is actually the result of the trajectory generated by the chaotic system in the high-dimensional phase space being projected into one dimension, this will cause the motion trajectory to be distorted during the projection process. In order to make the information extracted from the low-dimensional data incomplete and accurate, we need to choose the reconstruction dimension to recover the chaotic motion trajectory from the chaotic time series. The usual method of determining the embedding dimension in practical applications is to calculate some geometric invariants of the attractor (such as correlation dimension, Lyapunov exponent, etc.). From the analysis of Takens embedding theorem, it can be seen that these geometric invariants have the geometric properties of attractors. When the dimension is greater than the minimum embedding dimension, the geometric structure has been fully opened. At this time, these geometric invariants have nothing to do with the embedding dimension. Based in this theory, the embedding dimension when the geometric invariant of the attractor stops changing can be selected as the reconstructed phase space dimension.

Cao's method

In terms of determining the embedding dimension of the phase space, Cao False Nearest Neighbour Method (Cao's Method) [51] is considered to be one of the effective methods to calculate the embedding dimension. There are three advantages of using Cao's method to calculate the embedding dimension: First, it is only necessary to know the delay time parameter in advance. Second, only small amount of data is required. Third, random signals and deterministic signals can both be effectively identified.

Suppose the reconstructed data is:

$$X(t_i) = (x(t_i), x(t_{i+\tau}), \dots, x(t_{i+(m-1)\tau})), \quad i = 1, 2, \dots, N. \quad (\text{A.4})$$

Then the distance $R_{m(i)}$ between it and its nearest neighbor is

$$R_m(i) = \|X_m(i) - X_m^{FNN}(i)\|_2, \quad (\text{A.5})$$

where $X_m^{FNN}(i)$ is the nearest neighbor of $X_m(i)$.

Define

$$a(i, m) = \frac{R_{m+1}(i)}{R_m(i)} = \frac{\|X_{m+1}(i) - X_{m+1}^{FNN}(i)\|}{\|X_m(i) - X_m^{FNN}(i)\|}, \quad (\text{A.6})$$

when $X_{m+1}^{FNN}(i)$ and $X_m^{FNN}(i)$ are close, use the next adjacent point instead [56].

Next, calculate

$$E(m) = \frac{1}{N - m\tau} \sum_{i=1}^{N-m\tau} a(i, m), \quad (\text{A.7})$$

$$E_1(m) = E(m+1)/E(m). \quad (\text{A.8})$$

When the value of m keeps increasing and is greater than a certain value, the amplitude of the change of $E_1(m)$ can be neglected, resulting in saturation. At this time, the corresponding $m+1$ value is the best embedding dimension of the system.

Finally, we have

$$E^*(m) = \frac{1}{N-m\tau} \sum_{i=1}^{N-m\tau} |x_{i+m\tau} - x_{n(i,m)+m\tau}|, \quad (\text{A.9})$$

$$E_2(m) = E^*(m+1)/E^*(m). \quad (\text{A.10})$$

If for any m , $E_2(m) = 1$, it is a random time series, that is, each value of data is independent. If there is always some m , such that $E_2(m) \neq 1$, it is a deterministic time series, that is, the relationship between data points depends on the change of the embedding dimension m .

After calculating the maximum Lyapunov exponent, $E_1(m)$ and $E_2(m)$, we can identify the chaos and predictability of the sequence, and determine the reconstruction dimension. In this way, the dimension of the time series is restored to the original high dimension so that its potentially valuable features are extracted. At the same time, we also get the best input feature embedding for our model.

Appendix B

B1. Supplementary results for Section 4.2

In this appendix session, we have Fig. B.1 as supplementary results.

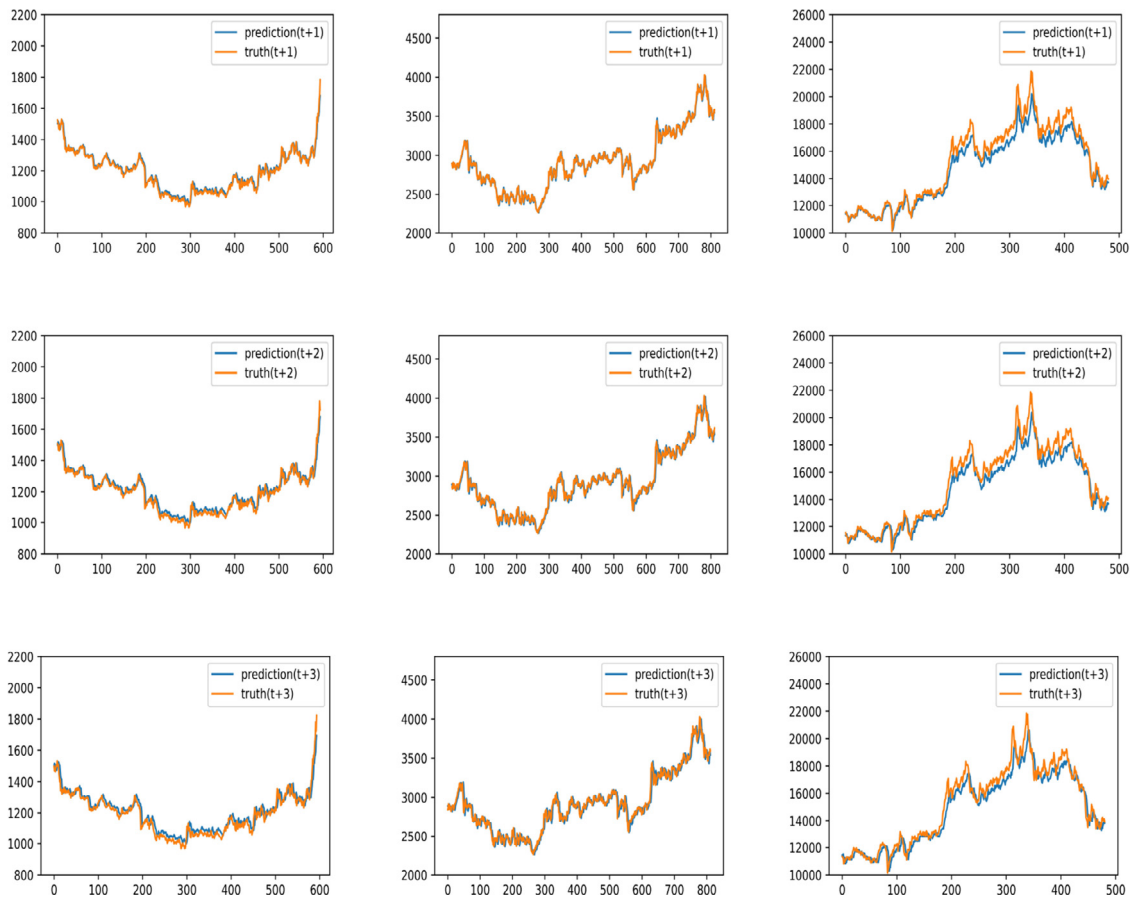


Fig. B.1. The prediction effect of different stocks with $\alpha = 1.5$. (left:SSE Energy Index, middle:SSE50 Index, right:SSE Consumer Index).

B2. Supplementary results for Section 4.3 (including)

In this appendix session, we have Fig. B.2 as supplementary results.



Fig. B.2. For SSE 50 Index, the predictive effects of different models. (The upper left is ARIMA, the upper right is LSTM, the lower left is SDE-Net, the lower right is LDE-Net.).

B3. Supplementary results for safe prediction horizon (Tables B.1 and B.2)

Table B.1

Forecast results of SSE Energy Index with different horizon ($\alpha = 1.5$).

Embedding Dimension	MSE(t+1)	MSE(t+2)	MSE(t+3)	MSE(t+4)
20	0.0010	0.0022	0.0034	0.0045
43	0.0013	0.0032	0.0058	0.0075
50	0.0018	0.0026	0.0048	0.0054

Table B.2

Forecast results of SSE50 Index with different horizon ($\alpha = 1.5$).

Embedding Dimension	MSE(t+1)	MSE(t+2)	MSE(t+3)	MSE(t+4)
23	0.0029	0.0058	0.0091	0.0125
41	0.0037	0.0070	0.0100	0.0137
50	0.0036	0.0066	0.0102	0.0138

B4. Supplementary results for different forecasting steps

In this appendix session, we have Table B.3, Figs. B.3, B.4 and B.5 as supplementary results for Section 4.5.

Table B.3

Forecast results of different steps ($\alpha = 1.5$).

Data	MSE(t+5)	MSE(t+6)	MSE(t+7)	MSE(t+8)	MSE(t+9)	MSE(t+10)
SSE Energy Index	0.0068	0.0071	0.0109	0.0140	0.0155	0.0168
SSE50 Index	0.0158	0.0188	0.0211	0.0236	0.0257	0.0285

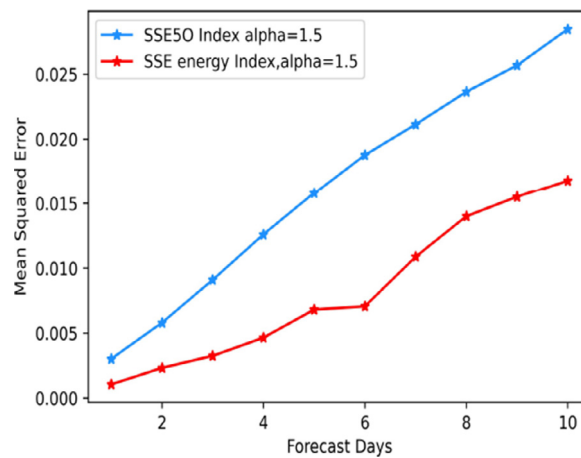


Fig. B.3. The prediction loss of SSE Energy Index and SSE50 Index with different steps. ($\alpha = 1.5$).

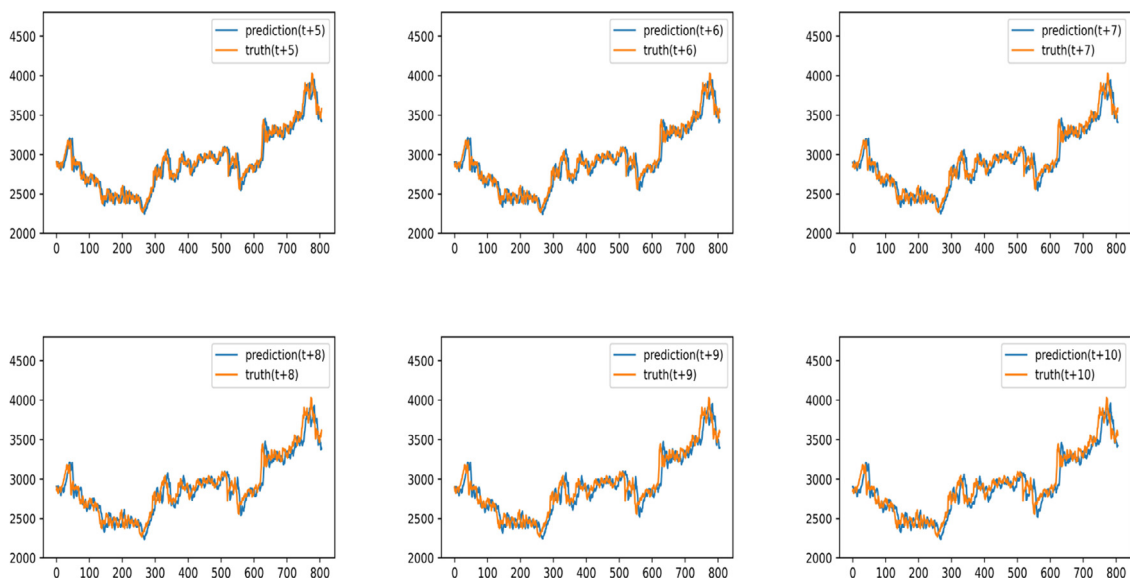


Fig. B.4. The prediction effect of SSE50 Index with different steps. ($\alpha = 1.5$).

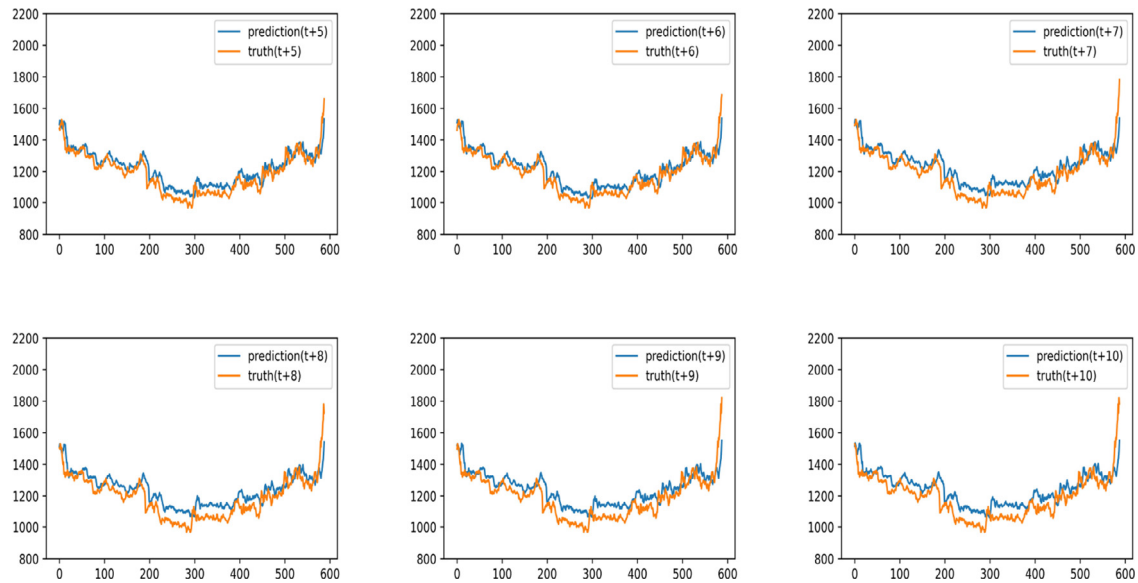


Fig. B.5. The prediction effect of SSE Energy Index with different steps. ($\alpha = 1.5$).

References

- [1] G. Hummer, Position-dependent diffusion coefficients and free energies from Bayesian analysis of equilibrium and replica molecular dynamics simulations, *New J. Phys.* 7 (1) (2005) 34.
- [2] D. Beltran-Villegas, T. Edwards, M. Bevan, Self-consistent colloidal energy and diffusivity landscapes in macromolecular solutions, *Langmuir* 29 (40) (2013) 12337–12341.
- [3] M. Sahoo, K. Patra, River water quality modelling and simulation based on Markov Chain Monte Carlo computation and Bayesian inference model, *Afr. J. Sci. Technol. Innov. Dev.* 12 (6) (2020) 771–785.
- [4] S. Sun, F. Jin, H. Li, Y. Li, A new hybrid optimization ensemble learning approach for carbon price forecasting, *Appl. Math. Model.* 97 (2021) 182–205.
- [5] C. Tian, Y. Hao, Point and interval forecasting for carbon price based on an improved analysis-forecast system, *Appl. Math. Model.* 79 (2020) 126–144.
- [6] X. Wu, Y. Wang, J. Mao, Z. Du, C. Li, Multi-step prediction of time series with random missing data, *Appl. Math. Model.* 38 (14) (2014) 3512–3522.
- [7] T. Zhong, New approach of short-term stock prediction based on combination of phase space reconstruction theory and recurrent neural network, *Appl. Res. Comput.* (2007).
- [8] K. Stergiou, T. Karakasidis, Application of deep learning and chaos theory for load forecasting in Greece, *Neural Comput. Appl.* 33 (23) (2021) 16713–16731.
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [10] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: beyond efficient transformer for long sequence time-series forecasting, *AAAI*, 2021.
- [11] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: decomposition transformers with auto-correlation for long-term series forecasting, *Adv. Neural Inf. Process. Syst.* 34 (2021).
- [12] Y. Lu, Z. Li, D. He, Z. Sun, B. Dong, T. Qin, L. Wang, T. Liu, Understanding and improving transformer from a multi-particle dynamic system point of view, *arXiv:1906.02762* (2019).
- [13] R. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud, Neural ordinary differential equations, *NeurIPS*, 2018.
- [14] X. Liu, T. Xiao, S. Si, Q. Cao, S. Kumar, C. Hsieh, Neural SDE: stabilizing neural ode networks with stochastic noise, *arXiv preprint arXiv:1906.02355* (2019).
- [15] J. Jia, A. Benson, Neural jump stochastic differential equations, *Adv. Neural Inf. Process. Syst.* 32 (2019).
- [16] L. Kong, J. Sun, C. Zhang, SDE-Net: equipping deep neural networks with uncertainty estimates, *arXiv:2008.10546* (2020).
- [17] M. Raissi, P. Perdikaris, G. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [18] L. Lu, X. Meng, Z. Mao, G.E. Karniadakis, DeepXDE: a deep learning library for solving differential equations, *SIAM Rev.* 63 (1) (2021) 208–228.
- [19] X. Chen, L. Yang, J. Duan, G.E. Karniadakis, Solving inverse stochastic problems from discrete particle observations using the Fokker-Planck equation and physics-informed neural networks, *SIAM J. Sci. Comput.* 43 (3) (2021) B811–B830.
- [20] J. O’Leary, J. Paulson, A. Mesbah, Stochastic physics-informed neural networks (SPINN): a moment-matching framework for learning hidden physics within stochastic differential equations, *arXiv preprint arXiv:2109.01621* (2021).
- [21] K. Kontolati, D. Alix-Williams, N. Boffi, M. Falk, C. Rycroft, M. Shields, Manifold learning for coarse-graining atomistic simulations: application to amorphous solids, *Acta Mater.* 215 (2021) 117008.
- [22] L. Gonon, C. Schwab, Deep ReLU network expression rates for option prices in high-dimensional, exponential Lévy models, *Finance Stoch.* 25 (4) (2021) 615–657.
- [23] J. Siegel, J. Xu, Approximation rates for neural networks with general activation functions, *Neural Netw.* 128 (2020) 313–321.
- [24] W. E, C. Ma, L. Wu, The Barron space and the flow-induced function spaces for neural network models, *Constr. Approx.* (2021) 1–38.
- [25] Q. Li, T. Lin, Z. Shen, Deep learning via dynamical systems: an approximation perspective, *arXiv preprint arXiv:1912.10382* (2019).
- [26] Q. Zhu, Asymptotic stability in the p th moment for stochastic differential equations with Lévy noise, *J. Math. Anal. Appl.* 416 (1) (2014) 126–142.
- [27] G. Sangeetha, K. Mathiyalagan, State estimation results for genetic regulatory networks with Lévy-type noise, *Chin. J. Phys.* 68 (2020) 191–203.
- [28] Y. Wang, H. Wang, S. Zhang, Prediction of daily PM_{2.5} concentration in China using data-driven ordinary differential equations, *Appl. Math. Comput.* 375 (2020) 125088.
- [29] C. Li, Y. Yang, S. Liu, A new method to mitigate data fluctuations for time series prediction, *Appl. Math. Model.* 65 (2019) 390–407.
- [30] Y. Lu, A. Zhong, Q. Li, B. Dong, Beyond finite layer neural networks: bridging deep architectures and numerical differential equations, 2018.

- [31] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [32] Z. Han, Y. Hu, C. Lee, Optimal pricing barriers in a regulated market using reflected diffusion processes, *Quant. Finance* 16 (4) (2016) 639–647.
- [33] L. Bo, Y. Wang, X. Yang, Some integral functionals of reflected SDEs and their applications in finance, *Quant. Finance* 11 (3) (2011) 343–348.
- [34] A. Browning, D. Warne, K. Burrage, R. Baker, M. Simpson, Identifiability analysis for stochastic differential equation models in systems biology, *J. R. Soc. Interface* 17 (173) (2020) 20200652.
- [35] G. Yuan, D. Ding, J. Duan, W. Lu, F. Wu, Total value adjustment of Bermudan option valuation under pure jump Lévy fluctuations, *Chaos* 32 (2) (2022) 023127.
- [36] F. Yang, Y. Zheng, J. Duan, L. Fu, S. Wiggins, The tipping times in an Arctic sea ice system under influence of extreme events, *Chaos* 30 (6) (2020) 063125.
- [37] W. Song, H. Liu, E. Zio, Long-range dependence and heavy tail characteristics for remaining useful life prediction in rolling bearing degradation, *Appl. Math. Model.* 102 (2022) 268–284.
- [38] Y. Zheng, F. Yang, J. Duan, X. Sun, L. Fu, J. Kurths, The maximum likelihood climate change for global warming under the influence of greenhouse effect and Lévy noise, *Chaos* 30 (1) (2020) 013132.
- [39] K. Kurumatani, Time series forecasting of agricultural product prices based on recurrent neural networks and its evaluation method, *SN Appl. Sci.* 2 (8) (2020) 1–17.
- [40] G.M. Rotskoff, E. Vanden-Eijnden, Trainability and accuracy of neural networks: an interacting particle system approach, *arXiv preprint arXiv:1805.00915* (2018).
- [41] J. Duan, *An Introduction to Stochastic Dynamics*, Cambridge Texts in Applied Mathematics, Cambridge University Press, 2015. <https://books.google.com.hk/books?id=jYWEBwAAQBAJ>
- [42] N. Ikeda, S. Watanabe, *Stochastic Differential Equations and Diffusion Processes*, Kodansha Scientific Books, North-Holland, 1989. https://books.google.com.hk/books?id=PUw_AQAAIAAJ
- [43] W. Liu, Strong convergence rate of Euler-Maruyama method for stochastic differential equations with Hölder continuous drift coefficient driven by symmetric α -stable process, *arXiv preprint arXiv:1901.08742* (2019).
- [44] G. Samorodnitsky, M. Taqqu, Stable non-gaussian random processes: stochastic models with infinite variance, *J. Am. Stat. Assoc.* 90 (1995) 805.
- [45] T.G. Kurtz, P. Protter, Weak limit theorems for stochastic integrals and stochastic differential equations, *Ann. Probab.* 19 (3) (1991) 1035–1070, doi:10.1214/aop/1176990334.
- [46] J. Ma, Y. Zhang, Z. Cao, Chaos characteristics of space object RCS based on Cao method, *Modern Radar* (2009).
- [47] A. Wolf, J. Swift, H. Swinney, J. Vastano, Determining Lyapunov exponents from a time series, *Physica D* 16 (3) (1985) 285–317.
- [48] T. Karakasidis, A. Charakopoulos, Detection of low-dimensional chaos in wind time series, *Chaos Solitons Fractals* 41 (4) (2009) 1723–1732.
- [49] J. Zbilut, Use of recurrence quantification analysis in economic time series, in: *Economics: Complex Windows*, Springer, 2005, pp. 91–104.
- [50] C. Yao, Q. Lin, Recurrence plots analysis of the CNY exchange markets based on phase space reconstruction, *North Am. J. Econ. Finance* 42 (2017) 584–596.
- [51] L. Cao, Practical method for determining the minimum embedding dimension of a scalar time series, *Physica D* 110 (1997) 43–50.
- [52] G. Frank, Recovering the Lyapunov exponent from chaotic time series, *Digitized Theses* (1990).
- [53] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Comput.* 9 (8) (1997) 1735–1780.
- [54] G. Box, G. Jenkins, G. Reinsel, G. Ljung, *Time Series Analysis: Forecasting and Control*, John Wiley & Sons, 2015.
- [55] F. Takens, Detecting strange attractors in turbulence, in: *Dynamical Systems and Turbulence*, Warwick 1980, Springer, 1981, pp. 366–381.
- [56] L. Li, *The Research of Financial Chaos based on Phase Space Reconstruction Technology*, Hunan University of Arts and Science (in Chinese), 2011.